JVD Internal Management System

Architecture & Database Design Review · v1.0 FINAL · 2026-07-04

JVD Architecture & Database Design Review

Date: 2026-07-04 · Part of the v1.0 doc set (see `IMPLEMENTATION_ROADMAP.md`) **Method:** inspection of all 140 migrations, 59 models, config, and the layering of `app/`

Verdict: the database design holds up better than expected — a solid B-minus foundation with four structural flaws, all fixable by additive migration. The application architecture is the right *kind* (layered monolith) but the layers leak. Nothing here requires a rebuild; everything slots into the existing phase plan.

---

1. What already meets professional standard (verified)

Practice	Evidence
Money stored as `decimal(15,2)` — never float	All financial columns; the single `float` in the schema is bus mileage (harmless; still convert)
Real foreign keys	125 `constrained()` FK declarations — referential integrity is mostly enforced by the DB, not by hope
Deliberate composite indexes on hot paths	e.g. `[bus_id, travel_date]` and `[driver_id, travel_date]` on the travel tables — exactly the right indexes for availability checks
Uniqueness on business keys	25 unique constraints (`invoice_number`, etc.)
Proper line-item normalization	`invoice_items`, `po_line_items`, `job_order_items` — parent/child modeled correctly, no CSV-in-a-column hacks
PostgreSQL	Right engine for this workload; supports the fixes below (partial unique indexes, check constraints)

This matters: the expensive-to-fix mistakes (float money, no FKs, denormalized blobs) were **not** made. What's wrong is fixable with migrations, not a re-platform.

2. The four structural flaws

2.1 The invoice became a god-table (root cause of the sales↔logistics↔accounting tangle)

`invoices` was created clean (number, customer, amounts, status) and then absorbed, via 8+ alter-migrations: `customer_name`, payment fields, downpayment fields, change fields, `bus_id`, `seat_map`, `travel_date`, driver linkage. **A financial document and an operational booking are now the same row.** That is *why* the sales→accounting handoff corrupts: logistics edits booking fields and accounting edits payment fields on the same record, each overwriting context the other needs (this compounds the no-transactions problem from `PRODUCT_IMPROVEMENT_SPEC.md` §9).

Fix (Phase 2): split into `invoices` (financial: immutable after finalization, per §9 rule 3) and `bookings` (operational: bus, driver, seats, dates, itinerary — mutable by logistics until trip completion), linked 1-to-n. Invoice totals derive from booked items at finalization and freeze. Each table gets exactly one writing module — §9 rule 1 becomes enforceable *because* the tables are separated.

2.2 Twin tables: `local_travels` / `international_travels` are byte-identical

Same columns, same indexes, two tables distinguished only by which one a row was inserted into. Every availability check must remember to query both (plus `pms_schedules`) — miss one and you double-book. The recent commit "Fix local vs international travel booking classification" is precisely the bug class this design guarantees.

Fix (Phase 2, small): one `travels` table with a `scope` column (`local|international`), backfill both tables in, add a **partial unique index** — `UNIQUE (bus_id, travel_date) WHERE status NOT IN ('cancelled')` — so the database itself makes double-booking impossible even if application locks fail (defense-in-depth for §9 rule 5). Same for `(driver_id, travel_date)`.

2.3 Cascade deletes on records that must never disappear

- `invoice_items` → `cascadeOnDelete`: hard-deleting an invoice silently erases financial history.
- `job_orders.passenger_id` → `cascadeOnDelete`: **deleting a passenger deletes job orders.**
- `local_travels.bus_id` → `cascadeOnDelete`: deleting a bus erases its travel history.

Only 7 tables have `softDeletes`. For an accounting-grade system the rule is: **operational/financial history is never hard-deleted** — finalized records get `restrictOnDelete` FKs (the DB refuses the delete), and "deleting" a bus/customer/passenger means soft-delete/deactivate.

Fix (Phase 2, one migration wave): flip cascades to `restrict` on all financial/operational children; add `softDeletes` to master data (buses, customers, passengers, suppliers, services, users already deactivatable).

2.4 Five dialects of informal polymorphism + 15 unconstrained IDs

`reference_type/reference_id`, `related_id`, `source_type/source_id`, `entity_id`, plus real Laravel `morphs` (used only 5 times) — five naming conventions for the same idea, most without indexes or any integrity guarantee. Plus 15 bare `*_id` columns with no FK at all (`invoices.bus_id`, `trip_tickets.bus_id`, `services.created_by`, `job_applications.converted_user_id`, `procurement_documents.transaction_id`, …) — these can silently point at deleted rows, which is one of the ways data gets "forgotten."

Fix: add the missing FKs where the target is a single table (most of the 15); standardize genuine polymorphism on Laravel `->morphs('reference')` naming with composite indexes; the §4 document repository's `document_links` table sets the pattern.

3. Smaller schema issues (fix opportunistically)

- **Status vocabulary drift:** `invoices.status` was created as `enum('pending','paid','cancelled')` but code now writes `draft`, `partial`, `pending_payment`, `balance`, `received` … 16 enum + 10 string status columns exist with no authoritative definition. The workflow engine (§5) becomes the single source of status truth;

as modules migrate onto it, convert DB enums to plain strings + a CHECK constraint generated from the workflow definition (PG enums are painful to alter; Laravel `enum()` is a CHECK anyway).
- **No CHECK constraints on amounts** (`total_amount >= 0`, `amount_received >= 0`) — one migration, cheap insurance.
- **`users` is three things at once** — login identity, HR employee (salary via `employee_salaries`), and operational resource (driver). Acceptable at this scale; if driver/HR attributes keep accreting, split `employee_profiles` off later. Not urgent.
- **80 of 140 migrations are alterations** — schema-by-accretion. Normal for a living app, but after Phase 2's structural fixes, run `php artisan schema:dump` to squash history into one baseline (faster CI, readable schema).
- **JSON columns (11)** are used reasonably (layouts, checklists, snapshots) — keep, but never query into them for business logic; anything filtered/joined on belongs in a real column.

4. Application architecture — does it hold up?

The shape is right, the discipline is missing. Layered monolith (routes → middleware → controllers → services → Eloquent) is exactly what a company-scale internal ERP should be (see Audit §5.3 — no microservices). The leaks:

1. **Layering is optional in practice** — business logic sits in 800-line controllers, services exist for only ~14 of 47 controllers, and there are two service namespaces. (Already Phase 2.9.)
2. **No module boundaries.** Any controller can touch any model — Sales writes booking fields, logistics reads invoice fields. The invoice/booking split (§2.1) plus "one writer per field" gives boundaries teeth. Optionally formalize later: group code as `app/Modules/{Sales,Accounting,Fleet,...}/{Controllers,Services,Models}` — a folder-move refactor, worthwhile only after Phase 2 stabilizes.
3. **Cross-module communication is direct writes; it should be events.** Accounting shouldn't be *called by* sales; it should *listen* for `InvoiceFinalized` and post the ledger entry itself (§9 pipelines). Laravel events (already needed for notifications §2) are the mechanism — same transaction via `afterCommit` semantics where required.
4. **The API layer is contract-less for half the modules** — `api-contracts/` covers 13; finish coverage as routes get versioned (Phase 2.8).

Target architecture in one sentence: a *modular monolith* — one Laravel app, one PostgreSQL database, hard module boundaries enforced by "one writer per field" + events for cross-module effects + the workflow engine for state + the ledger as the only money truth. This is the architecture NetSuite-class systems ship as, scaled to your team.

5. Roadmap insertions (these are now in `IMPLEMENTATION_ROADMAP.md`)

New item	Phase	Size
2.2b DB integrity wave: flip dangerous cascades → `restrict`, add missing FKs, CHECK constraints on amounts, softDeletes on master data	Phase 2, with 2.2	~2 days
2.2c Merge `local_travels` + `international_travels` → `travels` + partial unique indexes (DB-level double-booking guard)	Phase 2, with 2.2	~2 days
2.5b Split `bookings` out of `invoices`; invoice freezes at finalization	Phase 2, with snapshots (2.5)	~1 week, the biggest single fix
2.8b Status vocabulary: CHECK constraints generated from workflow definitions as modules migrate	Phase 2, rolling	absorbed
2.9b Cross-module effects move to events/listeners	Phase 2, with service extraction	absorbed
5.x `schema:dump` squash; optional `app/Modules/` reorganization	Phase 5	small

Bottom line: yes — it can be built to professional standard *on this foundation*. The schema's fundamentals (decimal money, real FKs, sane normalization, right indexes) are already professional; the four flaws are exactly the kind that additive migrations fix. The architecture needs discipline (boundaries, events, one writer per field) rather than replacement — and that discipline is what Phases 2's items install.